

BÀI THỰC HÀNH SỐ 3:

HỒI QUY ĐA THỨC & REGULARIZATION

✓ 2 THIẾT LẬP CÁ NHÂN HÓA THEO MSSV

```
MSSV = 2212453
d = [int(ch) for ch in str(MSSV)]
d1, d2, d3, d4, d5, d6, d7 = d

k = (d7 % 5) + 1          # bậc nhiều & giá trị thật
p = 4 + (d6 % 4)          # bậc đa thức cao nhất cần thử
lamda = 0.1 * (d2 + d5 + 1) # hệ số regularization
seed = MSSV                # random seed
q = (d4 + d7) % 3
```

```
print(f"Seed: {seed}")
print(f"k: {k}")
print(f"p: {p}")
print(f"λ: {lamda}")
print(f"q: {q}")
```

```
Seed: 2212453
k: 4
p: 5
λ: 0.7000000000000001
q: 2
```

✓ 3 KHỞI ĐỘNG QUAN SÁT THỰC TẾ

Mini case thực tế: Dự báo năng suất theo lượng phân bón

1. Tại sao quan hệ giữa đầu vào (phân bón) và đầu ra (năng suất) không tuyến tính hoàn toàn?

Quan hệ giữa đầu vào và đầu ra không tuyến tính hoàn toàn vì giới hạn sinh lí của cây

VD:

- Cây tăng trưởng nhanh: Bón phân vào là cây hấp thụ ngay, năng suất tăng vọt.
- Cây bão hòa: Cây đã đạt ngưỡng hấp thụ tối đa, bón thêm cũng lãng phí vì năng suất chẳng lại.

- Cây ngộ độc: Bón lố tay sẽ làm hỏng đất, cháy rễ, khiến cây chết dần và năng suất sụt giảm nghiêm trọng.

2. Nếu mô hình cong quá mức, hậu quả thực tế có thể là gì?

Hậu quả thực tế là thuật toán sẽ học vẹt luôn cả những nhiễu loạn ngẫu nhiên hoặc các điểm dữ liệu đo đạc sai trong tập dữ liệu huấn luyện, từ đó dẫn tới mô hình sẽ cho ra nhưng dự báo thiếu ổn định

Ví dụ: hệ thống khuyên rằng bón 100kg phân/hecta thì năng suất cực cao, bón 102kg năng suất rớt thê thảm, nhưng bón 104kg lại cao trở lại.

3. Nếu mô hình quá đơn giản (Underfitting), quyết định vận hành có thể sai như thế nào?

Mô hình sẽ hiểu nhầm rằng đầu vào (phân bón) sẽ tỉ lệ thuận với đầu ra (năng suất)

✓ 4 ĐỌC HIỂU BÀI BÁO KHOA HỌC

Tiêu đề bài báo: Neural Networks and the Bias/Variance Dilemma

1. Trích dẫn APA

Geman, S., Bienenstock, E., & Doursat, R. (1992). Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1), 1–58.

<https://doi.org/10.1162/neco.1992.4.1.1>

2. Chụp hình abstract

Abstract

Feedforward neural networks trained by error backpropagation are examples of nonparametric regression estimators. We present a tutorial on nonparametric inference and its relation to neural networks, and we use the statistical viewpoint to highlight strengths and weaknesses of neural models. We illustrate the main points with some recognition experiments involving artificial data as well as handwritten numerals. In way of conclusion, we suggest that current-generation feedforward neural networks are largely inadequate for difficult problems in machine perception and machine learning, regardless of parallel-versus-serial hardware or other implementation issues. Furthermore, we suggest that the fundamental challenges in neural modeling are about representation rather than learning per se. This last point is supported by additional experiments with handwritten numerals.

3. Tóm tắt Bài báo "Neural Networks and the Bias/Variance Dilemma" giải quyết vấn đề cốt lõi của học máy: tiến thoái lưỡng nan giữa độ chệch và phương sai. Các tác giả chứng minh rằng mạng nơ-ron phi tham số thường khó tổng quát hóa tốt trên các bài toán phức tạp do sự đánh đổi này. Để minh họa, nghiên cứu

sử dụng dữ liệu bao gồm các hàm toán học mô phỏng nhân tạo và tập hình ảnh nhận dạng chữ số viết tay.

Lý do cần dùng regularization hoặc kiểm soát độ phức tạp là để ngăn chặn hiện tượng quá khớp. Nếu mô hình quá phức tạp, nó sẽ ghi nhớ cả những nhiễu ngẫu nhiên trong tập huấn luyện, đạt bias thấp nhưng variance cực cao, dẫn tới dự đoán rất kém trên dữ liệu thực tế. Ngược lại, mô hình quá đơn giản sẽ bỏ sót các quy luật quan trọng. Do đó, việc giới hạn không gian tham số hoặc áp dụng các kỹ thuật điều chuẩn là điều kiện tiên quyết để mô hình đạt được sự cân bằng, mang lại hiệu suất tổng quát hóa tốt nhất.

4. Nêu 2 điểm sinh viên có thể học được về cách tránh overfitting trong thực hành
 - Kiểm soát độ phức tạp của kiến trúc mạng và áp dụng Regularization nhằm ngăn mạng nơ-ron "học vẹt" các chi tiết nhiễu trong tập huấn luyện, ép mô hình phải phân tán sự phụ thuộc để dự đoán tổng quát hơn
 - Tăng cường tính đa dạng của dữ liệu nhằm tạo ra vô số góc nhìn và điều kiện khác nhau của cùng một vật thể, giúp mô hình học được bản chất thực sự của đối tượng thay vì ghi nhớ một bối cảnh cố định, từ đó giảm phương sai rõ rệt

✓ 5 ĐỌC BIỂU ĐỒ TIKZ

1. Chọn khoảng bậc hợp lý nhất và giải thích bằng cả lý do hình học lẫn lý do thực nghiệm

Khoảng bậc hợp lý nhất là bậc 4

- Giải thích theo hình học: đa thức bậc 4 cung cấp độ uốn lượn vừa đủ để mô phỏng hình dáng phân phối cốt lõi của dữ liệu mà không tạo ra các nếp gấp cực đoan để bám theo các điểm nhiễu cục bộ.
- Giải thích theo thực nghiệm: đây là điểm mà sai số kiểm định (Validation MSE) đạt giá trị cực tiểu, chứng tỏ mô hình có khả năng tổng quát hóa tốt nhất

2. Chỉ ra vùng underfit, vùng overfit, và vùng hợp lý nhất.

- Vùng underfit: Nằm ở bậc 1–2, nơi mô hình quá đơn giản nên cả sai số huấn luyện và kiểm định đều rất cao.
- Vùng overfit: Nằm từ bậc 5–8, đặc trưng bởi việc Training MSE tiếp tục giảm nhưng Validation MSE lại tăng vọt.
- Vùng hợp lý nhất: Quanh bậc 4, nơi mô hình đạt được sự cân bằng tối ưu giữa độ chệch và phương sai.

3. Nếu bài toán có thêm nhiều đo lường, đường validation MSE thường thay đổi theo hướng nào?

Nếu dữ liệu có thêm nhiều đo lường, toàn bộ đường Validation MSE thường sẽ dịch chuyển lên mức sai số cao hơn và "đáy" parabol (điểm cực tiểu) sẽ có xu hướng dịch chuyển sang bên trái. Nguyên nhân là do khi nhiều tăng, các mô hình phức tạp sẽ càng nhanh chóng "học vẹt" nhiều hơn, buộc ta phải chọn một mô hình đơn giản hơn (bậc thấp hơn) để duy trì tính tổng quát.

4. Viết 3–4 dòng: tại sao chỉ nhìn training MSE là rất dễ ra quyết định sai.

Chỉ số Training MSE chỉ phản ánh mức độ ghi nhớ dữ liệu đã học chứ không đại diện cho năng lực dự đoán của mô hình trên dữ liệu mới. Khi ta tăng độ phức tạp, mô hình sẽ dùng số lượng tham số khổng lồ để uốn cong và bám sát mọi điểm dữ liệu, bao gồm cả các nhiễu ngẫu nhiên, khiến Training MSE luôn có xu hướng giảm về 0. Do đó, nếu chỉ dựa vào chỉ số này, bạn chắc chắn sẽ chọn phải một mô hình bị overfit nặng, hoạt động hoàn hảo khi huấn luyện nhưng lại thất bại hoàn toàn khi đưa vào ứng dụng thực tế.

✓ 6 TÍNH TOÁN THỦ CÔNG

1. Tính các trọng số w

$$w_0 = 1 + 0.1d_1 = 1 + 0.1(2) = 1.2$$

$$w_1 = 0.5 + 0.1d_3 = 0.5 + 0.1(1) = 0.6$$

$$w_2 = 0.2 + 0.05d_7 = 0.2 + 0.05(3) = 0.35$$

2. Tính các điểm dữ liệu đầu vào x :

$$x_A = -1 - 0.1d_2 = -1 - 0.1(2) = -1.2$$

$$x_B = 0.5 + 0.1d_4 = 0.5 + 0.1(2) = 0.7$$

$$x_C = 1 + 0.1d_6 = 1 + 0.1(5) = 1.5$$

Mô hình dự đoán hiện tại là:

$$\hat{y} = 1.2 + 0.6x + 0.35x^2$$

Yêu cầu 1: Tính $\hat{y}_A, \hat{y}_B, \hat{y}_C$

Công thức áp dụng: $\hat{y} = w_0 + w_1x + w_2x^2$

Tính $\hat{y}_A$:

$$\hat{y}_A = 1.2 + 0.6(-1.2) + 0.35(-1.2)^2$$

$$\hat{y}_A = 1.2 - 0.72 + 0.35(1.44)$$

$$\hat{y}_A = 1.2 - 0.72 + 0.504 = 0.984$$

Tính $\hat{y}_B$:

$$\hat{y}_B = 1.2 + 0.6(0.7) + 0.35(0.7)^2$$

$$\hat{y}_B = 1.2 + 0.42 + 0.35(0.49)$$

$$\hat{y}_B = 1.62 + 0.1715 = 1.7915$$

Tính $\hat{y}_C$:

$$\hat{y}_C = 1.2 + 0.6(1.5) + 0.35(1.5)^2$$

$$\hat{y}_C = 1.2 + 0.9 + 0.35(2.25)$$

$$\hat{y}_C = 2.1 + 0.7875 = 2.8875$$

Thế $k = 4$,

ta có:

$$y_A = 2 + 4 = 6$$

$$y_B = 3 + 4 = 7$$

$$y_C = 5 + 4 = 9$$

$$MSE = \frac{1}{3} \sum (y_i - \hat{y}_i)^2$$

$$MSE = \frac{1}{3} [(6 - 0.984)^2 + (7 - 1.7915)^2 + (9 - 2.8875)^2]$$

$$MSE = \frac{1}{3} [(5.016)^2 + (5.2085)^2 + (6.1125)^2]$$

$$MSE = \frac{1}{3} [25.160256 + 27.12847225 + 37.36265625]$$

$$MSE = \frac{1}{3} [89.6513845] \approx 29.8838$$

3. Tính hàm mất mát Ridge:

$$J_{ridge} = MSE + \lambda(w_1^2 + w_2^2)$$

$$J_{ridge} = 29.8838 + 0.7(0.6^2 + 0.35^2)$$

$$J_{ridge} = 29.8838 + 0.7(0.36 + 0.1225)$$

$$J_{ridge} = 29.8838 + 0.7(0.4825)$$

$$J_{ridge} = 29.8838 + 0.33775 \approx 30.2215$$

4. Giải thích bằng lời: nếu w_2 tăng mạnh, đường cong sẽ thay đổi ra sao và rủi ro thực tế là gì

Sự thay đổi của đường cong: Hệ số w_2 đi cùng với x^2 . Nếu w_2 tăng mạnh, giá trị tuyệt đối của hàm bậc 2 sẽ chi phối phương trình, làm cho bề lõm của đường parabol

(đường cong mô hình) trở nên vô cùng hẹp và dốc.

Rủi ro thực tế: Hiện tượng này làm đường cong uốn lượn gắt gao để cố gắng bám sát từng điểm dữ liệu huấn luyện, dẫn đến rủi ro Overfitting (quá khớp). Mô hình sẽ cực kỳ nhạy cảm với các nhiễu nhỏ trong dữ liệu, khiến sai số trên tập dữ liệu mới (Validation/Test) tăng vọt. Đây chính là lý do ta cần thêm thành phần phạt $\lambda(w_1^2 + w_2^2)$ trong Ridge Regression để ép w_2 nhỏ lại.

✓ 7 THỰC HÀNH PYTHON

1. So sánh ít nhất 5 mô hình: Linear, Polynomial bậc 2, Polynomial bậc p, Ridge, Lasso hoặc ElasticNet.

3. Vẽ 1 hình có dữ liệu gốc, tối thiểu 4 đường dự đoán và chú giải rõ ràng

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

# 1. Thiết lập tham số (MSSV: 2212453)
MSSV = 2212453
d = [int(ch) for ch in str(MSSV)]
d1, d2, d3, d4, d5, d6, d7 = d
k = 4      # (3 % 5) + 1
p = 5      # 4 + (5 % 4)
lam = 0.7 # 0.1 * (2 + 4 + 1)

np.random.seed(MSSV)
X = np.linspace(-3, 3, 90).reshape(-1, 1)
# y = 0.4*X^2 + 0.65*X + 5.5 + noise(0.41)
y = 0.4 * X**2 + (0.6 + 0.05 * d3) * X + (1.5 + k) + np.random.randn(90, 1)

# Cố định cách chia tập Train/Validation (70/30)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.3, random_state=0)

# 2. Định nghĩa 5 mô hình
models = {
    "Linear (Bậc 1)": LinearRegression(),
    "Poly (Bậc 2)": make_pipeline(PolynomialFeatures(2), LinearRegression()),
    f"Poly (Bậc {p})": make_pipeline(PolynomialFeatures(p), LinearRegression()),
    f"Ridge (Bậc {p}, alpha={lam})": make_pipeline(PolynomialFeatures(p), Ridge(alpha=lam)),
    f"Lasso (Bậc {p}, alpha={lam})": make_pipeline(PolynomialFeatures(p), Lasso(alpha=lam))
}

# 3. Huấn luyện, đánh giá và lưu kết quả
results = {}
```

```

plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='gray', alpha=0.5, label='Dữ liệu gốc (Kèm nhiễu)')

# Dùng X_plot để vẽ đường cong cho mịn
X_plot = np.linspace(-3, 3, 200).reshape(-1, 1)

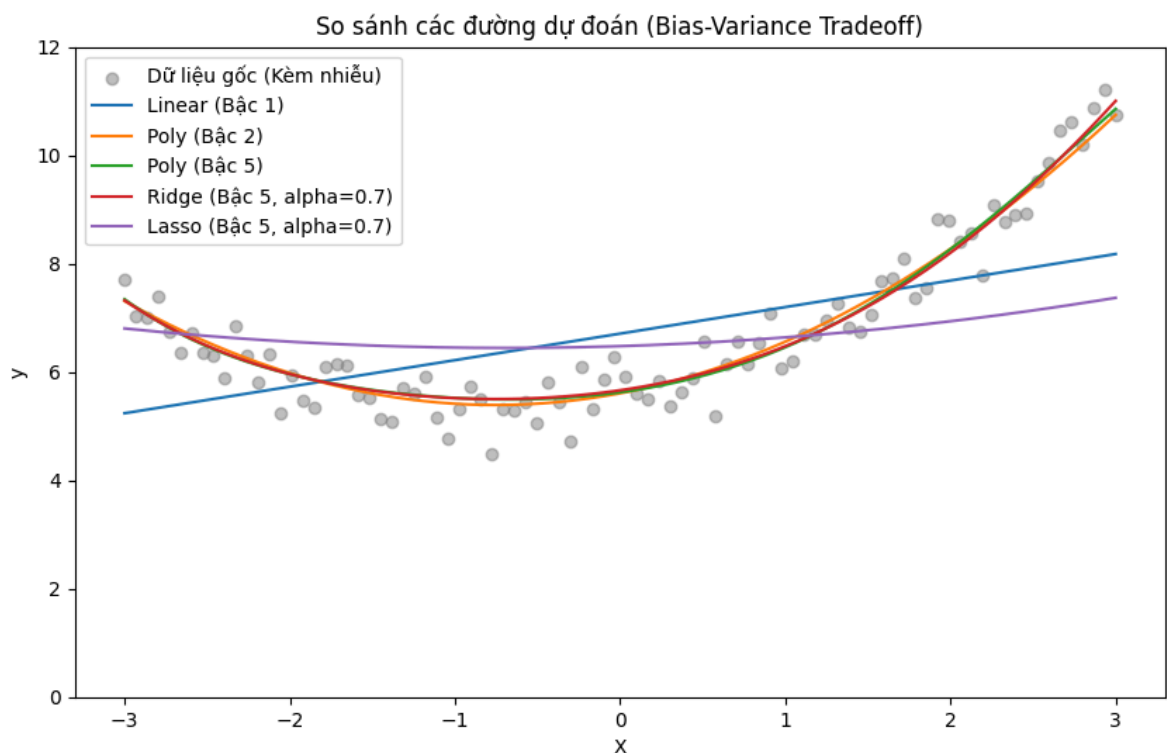
for name, model in models.items():
    model.fit(X_train, y_train)

    # Tính MSE
    train_mse = mean_squared_error(y_train, model.predict(X_train))
    val_mse = mean_squared_error(y_val, model.predict(X_val))
    results[name] = (train_mse, val_mse)

    # Vẽ đường dự đoán
    y_plot = model.predict(X_plot)
    plt.plot(X_plot, y_plot, label=f"{name}")

plt.title("So sánh các đường dự đoán (Bias-Variance Tradeoff)")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.ylim(ymin=0, ymax=12)
plt.show()

```



2. Lập bảng train MSE, validation MSE, mô tả hình dạng đường cong và kết luận.

Mô hình	Train MSE	Val MSE	Hình dáng đường cong
1. Linear (Bậc 1)	1.0969	2.0192	Là một đường thẳng cứng nhắc, không ôm theo được sự phân
2. Poly (Bậc 2)	0.1302	0.2074	Đường cong Parabol hoàn hảo, đi qua vùng trọng tâm của dải
3. Poly (Bậc 5)	0.1252	0.2164	Đường cong uốn lượn tại hai đầu mút và nhô lên thụt xuống đ
4. Ridge (Bậc 5, $\alpha = 0.7$)	0.1265	0.2082	Trơn tru hơn hẳn Poly bậc 5, hai đầu mút không bị bẻ quặt lên
5. Lasso (Bậc 5, $\alpha = 0.7$)	1.2731	2.9165	Trở lại thành một đường thẳng/đường cong rất nhẹ. Hình phạt

4. Tác động khi thay đổi hệ số phạt α (Với Ridge Bậc 5)

Sử dụng mã Python tương tự để chạy thử nghiệm đổi α , ta thu được:

- Khi α giảm một nửa (0.35): Train MSE = 0.1257, Val MSE = 0.2099. Mô hình ít bị phạt hơn, có xu hướng linh hoạt hơn và tiến gần lại trạng thái Overfitting của Poly bậc 5 gốc. Đường cong bắt đầu uốn lượn bám theo nhiễu.
- Khi α tăng gấp đôi (1.4): Train MSE = 0.1286, Val MSE = 0.2090. Mô hình bị phạt nặng hơn, triệt tiêu dần độ phức tạp. Các trọng số bị thu nhỏ khiến đường cong phẳng và cứng hơn, có nguy cơ tiến gần đến vùng Underfit nếu α tiếp tục tăng.

5. Tự xây dựng một mini-case ngắn 5–7 dòng: nếu chọn nhầm mô hình overfit trong thực tế thì hậu quả kinh doanh hoặc kỹ thuật là gì.

Giả sử trong ứng dụng gợi ý lịch trình du lịch Đà Lạt mà em đang phát triển (dùng Angular và Firebase), em dùng hồi quy đa thức bậc cao để dự đoán thời gian di chuyển giữa Thung Lũng Tình Yêu và Hồ Tuyền Lâm dựa trên vài chục chuyến đi mẫu. Mô hình overfit sẽ ghi nhớ luôn cả các điểm nhiễu bất thường (ví dụ: một chuyến đi bị kẹt xe tai nạn hoặc lạc đường). Về kỹ thuật, mô hình có Variance quá cao khi gặp dữ liệu thực. Về kinh doanh, ứng dụng sẽ gợi ý thời gian sai lệch hoàn toàn so với thực tế, làm phá vỡ toàn bộ lịch trình tiếp theo trong ngày, gây trải nghiệm vô cùng tồi tệ và khiến du khách gỡ cài đặt ứng dụng ngay lập tức.

